

Quick Vim Reference

Kurt Schmidt

May 2025

Contents

Foreword	4
Crutches	4
1 Modes	5
1.1 Getting Lost	5
2 Startup	6
2.1 Some Handy Settings	6
2.2 Options Beginners Might Consider	7
2.3 .vimrc – Vim Config File	8
3 Getting Help	9
3.1 Signalling Context	9
4 Administrative Commands (Normal Mode)	10
4.1 Background	10
4.2 Redrawing the Window	10
5 Motion	11
5.1 Line Numbers	11
5.2 Searching	12
6 Insert Mode	13
6.1 Inserting Literal Character <code>i_ctrl-v</code>	13
6.2 Unicode Characters	13
7 Edit Operations	14
7.1 Undo / Redo / Repeat	14
7.2 Deleting Text	14
7.3 Pasting Text (Put)	14
7.4 Copying Text (Yank)	15
7.5 Changing Text	15
7.6 Registers	16
7.7 Repetition	16
8 Formatting	17
8.1 Indent/Exdent	17
8.2 Joining 2 lines	17
8.3 Re-formatting	17
9 Vim Text Objects	18
9.1 Source Code Objects	18
10 Marks, Visual Mode	19
10.1 Visual Mode	19
11 Splitting the Window	20
11.1 Managing Windows	20
11.2 Moving Between Windows	20
11.3 Command History Window	20
12 Buffers	21

12.1	Buffers v Tabs	21
12.2	Using Multiple Buffers	21
12.3	Working With Buffers	21
13	Command Mode	22
13.1	Some Really Helpful Commands	22
13.2	Ranges	23
13.3	External Filters	24
14	Folds	25
15	Odds and Ends	26
	Epilogue	27

Foreword

This document is a quick reference, for things you've learned, and a quick survey of some features you might wish to learn.

You must go through the Vim tutorial first

```
$ vimtutor
```

, and keep practising. Get comfortable, and add new skills you find useful.

Crutches

While learning Vim I *really* discourage the use of the mouse, and of the arrow keys. Navigation, selection, will take a little bit of practice, but, this effort is well worth it. There is much more available than the four simple motions, but you'll never get there if you stop at the arrow keys.

The mouse is great for editing photos, playing solitaire, but moving the cursor, selecting text, we have many fine tools. Similarly, the mouse will slow you down.

1 Modes

Vim typically starts in *Normal Mode*. You can navigate around your document, save it, go into insert mode, etc. Usually the <ESC> key will bring you back to Normal Mode.

Some of the other modes:

Insert Mode Where you do all your typing. See [Insert Mode Section](#)

Visual Mode Use motions to visually select text. See [Visual Mode Section](#)

Replace Mode Really just an overstrike mode, as best I can tell.

Use <ESC> to get back to *Normal Mode*, same as *Insert*

Command Mode For things that start with a :, /, or ?. Shows up at the bottom of the window.

See also the [Command History Window](#)

1.1 Getting Lost

<ESC> will very often cancel the current command, *Normal Mode* key sequence, etc., or, return you to *Normal Mode*.

If you find yourself in **ex** (line editor) mode, try `:vim`.

2 Startup

Some useful features can be invoked at startup, either through a command-line option, or installed alias:

view Starts vim in read-only mode. Same as `vi -R`

vimdiff Starts you in a diff mode, documents are side-by-side. Make your terminal wide. `vim -d`

gvim gview The GUI version (if installed). `vim -g`

2.1 Some Handy Settings

The `set` command is used to see or change many vim settings. There are three types:

- number
- string
- toggle

To see summary settings:

```
:set
```

To see all:

```
:set all
```

To view a particular setting:

```
:set filetype?
```

2.1.1 Setting Number/String Options

```
:set ft=python
:set tabstop=3
:set verbosefile=
```

2.1.2 Setting a Toggle Option

Turn it off by prepending the option with “no”.

```
:set hlsearch
:set nohlsearch
:set number
:set nonumber
```

2.2 Options Beginners Might Consider

Note, you often needn't spell out the entire option. I'll include common shortcuts that I remember. Examples will show settings I recommend.

[no]compatible cp Don't behave as the original VI editor

```
set nocompatible
```

hidden hid Allows you to switch to another buffer w/out saving current one

```
set hidden
```

background bg By default, color schemes assume a light background. If you use a dark terminal, then:

```
set bg=dark
```

showcmd sc Toggle. Show partial commands (what you're typing in *Normal Mode*) at bottom

ruler ru Toggle. Shows row and column in lower right corner

hlsearch hls Toggle. Highlights matches from a search. (Use `:set nohl` to turn it off, or see below for a binding I use)

incsearch is Toggle. Show matches as regex is being typed

wildmenu Toggle. Better command-line completion (try the <Tab> key)

showmode smd Toggle. Displays the vim mode, bottom left (I think)

showmatch sm Toggle. Shows partner when cursor is placed on a bracket (parenthesis, etc.).

textwidth tw Number. Width of buffer. Where VIM will insert characters (on whitespace), as you type (format cmd, etc.)

tabstop ts Number. The number of columns a tab displays. (Does *not* affect the buffer, only the appearance.)

```
set tabstop=3
```

shiftwidth sw Number. Number of spaces to use for indent, shift (<<, >>, etc.). Set to 0 to use tabstop value

```
set shiftwidth=3
```

- Set to 0 to use value of **tabstop**

```
set shiftwidth=0
```

expandtab et Toggle. Typed tabs will be converted to **tabstop** spaces in buffer

- Doesn't affect existing tabs
- See `:retab`

number nu Toggle. Turns on line numbers.

I find I don't really use this. I have the ruler, and <n>G will take me to a particular line.

relativenumber ru Toggle. Give it a try. I don't use it, but, some might find it handy, especially when **nu** is also set

wrap Toggle. Display only. Long lines are wrapped, rather than disappearing off to the right. Doesn't modify the buffer.

showbreak sb String. Prefix for wrapped lines.

2.3 .vimrc – Vim Config File

You can put your customisations in a config file. Vim will read `~/.vimrc` upon startup. (GVim will instead read `~/.gvimrc`, if present.)

I have created a simple starter `.vimrc` file for newbies to use. Users on the department machines should copy it from my `Public` directory. Or, [you can look at it here](#).

Note, the double-quote, `"`, introduces a line comment in this file

Other than the options discussed above, here are some other settings you might find helpful:

syntax on Turns on syntax highlighting, autoindenting, etc. (as appropriate)

map Y y\$ Behave like `y$`, rather than `yy`

D and C work in this way. The original Vi was inconsistent, here.

nnoremap <C-L> :nohl<CR><C-L> Ctrl-L now turns off search highlighting, then redraws the screen

map <F1> <Esc>

imap <F1> <Esc> I got tired of hitting the `<F1>` (help) key, when reaching for the `<ESC>` key

[Older Lenovos had `<ESC>` *above* the `<F1>` key.]

set pastetoggle=<F2> Use `<F2>` to toggle between `paste` and `nopaste`. Helpful if pasting into a buffer from the terminal or desktop clipboard

map <F3> :set invlist<CR> Use `<F3>` to toggle between `list`, which shows you newlines, tabs, and other characters, and `nolist`

map <Leader>l :up<CR>:!pdflatex %<CR> Type `\l` (in normal mode) to save the current buffer, run TexLive's `pdflatex` on it

map <Leader>p :up<CR>:!pandoc --highlight=tango % -o %:r.pdf<CR> Type `\p` (in normal mode) to save the current buffer, run `pandoc` on it, converting `file.md` to `file.pdf`.

map <Leader>t :up<CR>:!tcc -run %<CR> Type `\t` to save current buffer, run it in `tcc` (interpreter mode)

map <Leader>y :up<CR>:!python3 %<CR> Type `\y` to save current buffer, run it in `python3`

map <Leader>m :wall<CR>q:??^make<CR><CR> Type `\m` to save any dirty buffers, then re-execute the most recent `make` command

These are just examples of common tasks I use. The last, `make`, is most versatile. You make your own.

Note, VIM replaces `%` in a command with the current filename. `:up` is “update”, it saves the current buffer (if dirty). `:!` runs an external program.

3 Getting Help

From normal mode:

```
:help
```

It takes an optional argument. Wildcards and tab-completion (or CTRL-D) might help you. E.g.,

```
:help k
```

To exit help:

```
:helpclose
```

, or, if it's the current frame:

```
:q
```

Jump to specific topics by putting cursor on highlighted tag, hit `ctrl-]`. Use `ctrl-o` or `ctrl-t` to jump back.

To review help navigation:

```
:help :help
```

3.1 Signalling Context

Strings, commands, topics, might appear in several contexts. E.g., if you enter `:help list` you'll get information on the list container in Vi Script, rather than, e.g., a discussion of the `list` option which shows tab characters.

Prefix	Example	Context
<i>none</i>	<code>:h r</code>	Normal mode
<code>v_</code>	<code>:h v_r</code>	Visual mode
<code>i_</code>	<code>:h i_CTRL-W</code>	Insert mode
<code>‘:</code>	<code>:h :r</code>	Command-line command
<code>c_</code>	<code>:h c_ctrl-r</code>	Command-line editing
<code>-</code>	<code>:h -r</code>	Vim command argument (starting Vim)
<code>'</code>	<code>:h 'ro'</code>	Option
<code>/</code>	<code>:h /\r</code>	Regular expression

4 Administrative Commands (Normal Mode)

<ESC>	Cancels current command, or, returns you to <i>Normal Mode</i>
:e fn	Edit fn in a new buffer
:q	Close current window (if no changes)
:q!	Close current window, dammit! (Lose changes)
:w [fn]	Write buffer to <i>fn</i> , if provided; otherwise, use current buffer name
:w! [fn]	Write buffer, possibly overwriting target
:wq	Write and quit
ZZ	Write and quit
:x	Write (only if dirty) and quit

4.1 Background

If you use a terminal with a dark background:

```
:set bg=dark
```

4.2 Redrawing the Window

ctrl-L	Redraw (refresh) the screen
z<CR>	Redraw, current line to top of window
z.<CR>	Redraw, current line to center of window
z-<CR>	Redraw, current line to bottom of window

5 Motion

You can move the cursor around with the following keystrokes while in *Command Mode*. Most (all?) of the following can be preceded by a count. Default is 1.

Simple motions

h	Left
j	Down
k	Up
l	Right
0	Beginning of line
\$	End of line
H	Top of window
M	Middle line of window
L	Bottom of window
gg	1 st line of file
nG	<i>n</i> th (last) line of file
ctrl-f	Forward a screenful
ctrl-b	Backward a screenful

Please don't use the arrow keys. They're slower than the 4 basic motions, right on the home row. Get good at the basic motions, then add other motions. Don't just dead-end at arrow keys. That's Notepad.

Motions over objects

w , W	Beginning of next (big) word
e , E	End of next (big) word
b , B	Back. Beginning of previous (big) word
(,)	Beginning (end) of this or previous (next) sentence
{ , }	Beginning (end) of this or previous (next) paragraph

5.1 Line Numbers

- To see your line, column and apparent column in lower right:

```
:set ruler
```

- To go to a particular line number **nn**:

```
nnG
```

- If you really want to see line numbers:

```
:set number
```

To turn it off:

```
:set nonumber
```

Also, try this:

```
:set relativenumber  
:set nornu
```

5.2 Searching

<i>/re</i>	Forward incremental regex search
<i>?re</i>	Backward incremental regex search
<i>n N</i>	Go to next (previous) match, same direction
<i>#</i>	Match previous string under cursor
<i>*</i>	Match next string under cursor
<i>f char</i>	Advance cursor to next <i>char</i> on current line
<i>F char</i>	Move cursor left to previous <i>char</i> on current line
<i>t char</i>	Advance cursor to 1 column before next <i>char</i> on current line
<i>T char</i>	Move cursor left to 1 column after previous <i>char</i> on current line
<i>;</i>	Repeat the previous <i>f</i> , <i>F</i> , <i>t</i> , or <i>T</i> command
<i>''</i>	Return cursor to previous line

Note, */* and *?* are *Command-line Mode* commands; the cursor will jump to the bottom of the window.

6 Insert Mode

There are various ways to get into *Insert Mode* from *Command Mode*:

i	Insert at (prior to) cursor
I	Insert at first (non-white) character in the line
a	Append at next column after cursor
A	Append at end of current line
o	Open new line below cursor
O	Open new line above cursor

Each of these can be preceded by a count. You type some text. When you hit <ESC>, the edit is repeated.

6.1 Inserting Literal Character `i_ctrl-v`

Use `ctrl-v` in *Insert Mode* (`i_ctrl-v`), or *Command Mode*, to quote a literal character, like, e.g., Backspace.

Handy if you want the replacement string in a substitute command to have a Newline in it:

```
%s/, /~M/g
```

You can't type `^M` as 2 characters; instead, you hit `ctrl-v` while typing, then the `Enter` key.

6.2 Unicode Characters

6.2.1 Enter a Unicode

From *Insert Mode*, hit `ctrl-v + u + code`. E.g., use `u2713` to insert a check mark.

6.2.2 Digraphs

Vim maps a great many 2-character (and 3-character) sequences to special characters.

While typing in *Insert Mode*, use `ctrl-k`, then the 2-key combo.

To see a list of digraphs, with descriptions:

```
:help digraphs-default
```

. May be useful to search (`/re`) for a character.

To see a complete, more compact table:

```
:digraph
```

7 Edit Operations

7.1 Undo / Redo / Repeat

u	Undo
ctrl-r	Redo
.	Repeat the last edit

Unlike in the original Vi, we have an undo (and a redo) stack, so, you can undo all the way back to the loaded buffer. (See variable `undolevels`.)

7.2 Deleting Text

These commands are available in *Normal Mode*, and can be preceded by a count:

x	Delete character under the cursor
dmotion	Delete text from cursor to point of <i>motion</i>
dd	Delete current (and following, if <i>count</i> > 1) lines
D	Delete from cursor to end of line

7.3 Pasting Text (Put)

To paste deleted or yanked text into a document from *Normal Mode*

P	Paste (before cursor)
p	Paste (after cursor)

xp is a quick, handy way to transpose two characters

7.3.1 Pasting from Outside Vim

Note about pasting from an external clipboard: Things like autoindent, etc., may cause problems if you paste already formatted text or code into a document. Vim supplies a *paste* setting, which turns off much of the autoformatting:

```
:set paste
```

Remember to cancel this setting for regular editing when done:

```
:set nopaste
```

With this setting in my `.vimrc` I just hit the key to toggle on and off:

```
" Use <F2> to toggle between 'paste' and 'nopaste'
set pastetoggle=<F2>
```

7.3.2 Read a File, or Command Output, into Buffer

I often find it handy to skip the clipboard, to simply read contents from a file `<fn>` right into the buffer. From *Normal Mode*:

```
:r <file>
```

, will insert contents starting at line beneath the cursor.

To place output from a command into buffer, e.g.:

```
:r! grep Waldo *.tex
```

```
:r! date -R
```

```
:r! head -n12 <file>
```

7.3.3 Open Source in New Buffer

Or, open the file in a new **buffer**, use Vim to yank text from there to another buffer.

7.4 Copying Text (Yank)

<i>ymotion</i>	Yank (copy) word, sentence, paragraph, etc.
<i>cntyy</i>	Yank (copy) <i>cnt</i> line(s)
Y	Yank (copy) <i>cnt</i> lines
y\$	Yank (copy) to end of line

7.5 Changing Text

r	Replace single character under the cursor with another. Leaves you in <i>Normal Mode</i>
R	Puts you in <i>Replace Mode</i>
c	<i>Change</i> . Used much like d , above, but leaves you in input mode
s	<i>Substitute</i> Delete char under cursor, leaves you in <i>Insert Mode</i>
~	Changes case of character under cursor. Can be used w/a count, or highlighted text

7.5.1 Changing Case

Each of these is followed by a motion or text object

gU	Change text to upper-case
gu	Change text to lower-case
g?	Performs Rot13 encoding (decoding)

7.6 Registers

Any of the lower-case letters can be used to store deleted or yanked (copied) text, when preceded by a `"`. Simply precede any delete, yank, or put operation with a register specifier:

```
"f2yy # copy current line and line below into register f
# move elsewhere
"fp # paste the 2 lines after the cursor
"fP # paste the 2 lines before the cursor
```

You can use *Visual Mode* when yanking or deleting text.

The numbered registers contain a history of your yanks. Vim maintains other registers, too.

:reg Will show you all the registers and their contents

Vim uses the numbered registers, 0-9, and a few others, to automatically store yanked/deleted text and other things.

```
"%p # paste filename of current buffer into the buffer
```

7.7 Repetition

Many edits and other commands might be preceded by a *count*. (You'll see the count appear in the lower right.) So:

```
12j Moves the cursor down 12 lines
20oword[ESC] Prints word on its own line 20 times
80a-[ESC] Makes a divider line containing 80 hyphens
```

If you accidentally enter a count, remember, `[ESC]` cancels the current command.

8 Formatting

8.1 Indent/Exdent

>>	indent line
<<	exdent line

- As with many commands, can be preceded by a count
- Or, use *Visual Mode*, then a single < or > to move whole section
- Or, < followed by a *motion* or a text object

8.2 Joining 2 lines

Use J to join the current line with the line below.

8.3 Re-formatting

You can ask Vim to reformat, e.g., a paragraph, or a function

- Use *Visual Mode*, then gp
- Or, gp<motion>
 - gp} – reformat to end of paragraph
- Inner-objects
 - gpip – reformats entire paragraph cursor is on
 - (See [Text Objects](#), below)

8.3.1 Re-do Tabs

Changing a tab setting (**ts**, **expandtabs**, etc) doesn't affect tabs already in the buffer.

:retab

will help you here. Takes an optional range:

:5,\$retab

to retab lines 5-end.

9 Vim Text Objects

These *Normal Mode* keystrokes are sorta like motions, but, they don't generally use the location of the cursor as a mark. E.g., if the cursor is anywhere in a word, we can delete the word, or the sentence, or reformat the paragraph.

Each object can be selected with (a) or without (i) surrounding whitespace (sorta).

aw	A word
iw	An inner word
aW iw	A (inner) big word
as is	A (inner) sentence
ap ip	A (inner) paragraph

9.1 Source Code Objects

I don't know that the behavior is dependent on filetype, I've not played, but, these objects make more sense in the context of code, some of them language-dependent.

9.1.1 Strings

a" i"	A (inner) double-quoted string
a' i'	A (inner) single-quoted string
a` i`	A (inner) back-quoted string

9.1.2 Other Bracketed Expressions

a) i)	A (inner) parenthesized block
a] i]	A (inner) bracketed block
a} i}	A (inner) brace block
at it	A (inner) XML-like markup tag block
ai ii	A (inner) indented block

9.1.3 Examples

dis	Delete sentence cursor is on
>ip	Indent current paragraph
gqip	Reformat (re-wrap) current paragraph
gUiw	Make current word all uppercase
guit	Make current (XML) markup tag block all lowercase

10 Marks, Visual Mode

Marks can be used for navigation, and for commands that take ranges. 2 marks can delimit a selection of text, for a variety of actions, including deleting/yanking, reformatting, write (:w), substitute (:s) and filter (:!) commands.

- To set a mark: `mc` , where `c` is any lower-case letter
- To go to that line: `'c` , where `c` is the mark

10.1 Visual Mode

Visual Mode allows you to graphically select text, sets two marks for you: ``<` and ``>`. While in *Visual Mode*, *Normal Mode* commands (indent, delete, yank, etc.) will act on this text, and *Command Mode* commands (`:w`, `:s`, etc.) will appear w/the range marks already supplied.

To highlight text, go to one end, then hit:

<code>v</code>	visual
<code>V</code>	visual, line mode
<code>ctrl-v</code>	visual, block mode

, the use motion keys (or a search) to get to the other border.

`<ESC>` will cancel the current *Visual Mode*, return you to *Normal Mode*.

Example:

- Use *Visual Mode* to highlight several lines of text
- Type
:
 - Note the bottom line of your screen. The special marks are already filled in
- Continue to type
`w ~/tmp/blah.sav`
 - Note, you've save the highlighted text to a new file
 - Remember to delete the file
- Or, hit `<ESC>` to cancel the command

11 Splitting the Window

[I don't quite have the jargon correct, so, don't get too formal.]

When you call for `:help`, e.g., the window splits into 2 panes.

Other ways to split the window / open a new one (from *Normal Mode*) :

<code>:spl</code>	Splits window horizontally
<code>:spl <fn></code>	Opens new window horizontally, opens file
<code>:vspl</code>	Splits window vertically
<code>:vspl <fn></code>	Opens new window vertically, opens file
<code>ctrl-w n</code>	Opens new window horizontally
<code>ctrl-w v</code>	Splits window vertically
<code>:term</code>	Opens new window above, starts a shell instance
<code>q:</code>	Opens the Command History Window

11.1 Managing Windows

Again, many of these can be preceded by a count, which often determines window's size (height or width)

<code>:q[!]</code>	Quit current window [dammit!]
<code>ctrl-w c</code>	Close current window
<code>ctrl-w o</code>	Make current window only (close others)
<code>ctrl-w +/-</code>	Increase/Decrease height of window
<code>ctrl-w >/<</code>	Increase/Decrease width of window
<code>ctrl-w H/L</code>	Move window left/right
<code>ctrl-w J/K</code>	Move window down/up

11.2 Moving Between Windows

<code>ctrl-w h</code>	Go <i>n</i> windows left
<code>ctrl-w l</code>	Go <i>n</i> windows right
<code>ctrl-w j</code>	Go <i>n</i> windows down
<code>ctrl-w k</code>	Go <i>n</i> windows up

11.3 Command History Window

This lets you search back through your command history, perhaps edit, and re-execute commands. `q:` will open a small (7-line) window, which you can navigate in the normal way.

Hit `[ENTER]` to execute the command the cursor is on.

`:q` or `ctrl-C [ENTER]` to cancel.

12 Buffers

12.1 Buffers v Tabs

What you think of as “tabs” are “buffers” in Vim. In short, Vim “tabs” should be called “layouts”. Think of Vim’s buffers as you might regard tabs in a different program.

[Click here](#) for a discussion of “Tab Madness”.

12.2 Using Multiple Buffers

While editing a file you can use `:e <fn>` to open another file into a new buffer.

12.3 Working With Buffers

<code>:ls</code>	List (and index) buffers
<code>:b <n></code>	Switch to buffer -
<code>:bd</code>	Delete (close) current buffer (if no changes)
<code>ctrl-6</code>	Switch between 2 most recent buffers

I have this handy binding in my `.vimrc`, so I hit `gb`, the buffers are listed, then I just type the new buffer number:

```
noremap gb :ls<CR>:b<Space>
```

Careful, `:q` will try to quit all the buffers.

13 Command Mode

Largely, everything can be done here. The original Vi was built on an older line-editor, Ex. You couldn't move the cursor around the document, so, everything was done from this command line. It is here you save the buffer, open new ones, quit, change settings, delete or replace text, delete lines, insert or append, run the buffer against an external filter, etc.

You already know some:

```
:w[!]  
:q[!]  
:wq  
:x  
:r  
:set
```

13.1 Some Really Helpful Commands

13.1.1 Delete

Without a range, deletes the current line:

```
:d
```

13.1.2 Substitute

Does a REGEX search and replace on first occurrence of current line:

```
:s/white chocolate/not chocolate/
```

To replace *every* occurrence on the line:

```
:s/this/that/g
```

13.1.3 Global

Allows us to apply an Ex command only to lines that match some regular expression.

To delete empty lines:

```
:g/^$/d
```

To delete blank lines (maybe contain whitespace):

```
:g/^\s*$/d
```

13.2 Ranges

Many of these commands which act on the buffer can be preceded by a range (the default is simply the current line).

Ranges can be delimited by:

- Line numbers
 - `.` is the current line
 - `$` can be used for the last line
 - `%` is a range, the entire file (short for `1,$`)
- Marks
- Regular Expressions
 - REs are delimited by `/`

Some examples:

- To delete lines 1-12:
`:1,12 d`
- To swap first occurrence of Merry for Pippin on each line, from current line to end:
`:$s/Merry/Pippin/`
- To swap every occurrence of Merry for Pippin on each line, from current line to end:
`:$s/Merry/Pippin/g`
- To save the HTML header section to another file:
`:/<head>/,/<\head>/ w ~/public_html/general_header.html`
 - Note, had to escape that inner `/`
- Make a fold out of the highlighted code (*Visual Mode*):
`:`<,>fo`
- Given 2 marks `a` and `b`, `a` precedes `b`, comment out all lines in range (inclusive)
`:`a,`bs/^/# /`

We can mix range element types.

- Delete lines from beginning of file until the string `<body>` is found (inclusive):
`:1,/<body>/! d`

13.2.1 Offsets

We can add or subtract lines from range indicators. E.g., to delete starting 2 lines after the cursor up to (and including) the line before the next occurrence of “Waldo”:

```
:.+2,/Waldo/-1 d
```

13.3 External Filters

You can apply any external program which reads `stdin` and writes to `stdout` (filter) to the buffer.

To sort all lines of the file:

```
:%! sort
```

Use `awk` to put line numbers in the file:

```
:%! awk '{printf( "%3s %s\n", NR, $0 )}'
```

- We had to escape the `%` from Vim
 - Vim will replace `%` with the filename of the current buffer

14 Folds

The easiest way to create a fold is to use visual mode. Highlight section, then:

`:fo`

- or:

`zf`

Of course, text objects work. To fold the current paragraph:

`zfiP`

Then, we can open, close, toggle, and delete folds:

<code>zd</code>	Delete one fold at the cursor
<code>zD</code>	Delete folds recursively at the cursor
<code>zo</code>	Open one fold under cursor
<code>zO</code>	Open all folds under cursor recursively
<code>zc</code>	Close one fold under cursor
<code>zC</code>	Close all folds under cursor recursively
<code>za</code>	Alternate (open closed fold, close open fold)

15 Odds and Ends

Here are commands and options which I use often enough, but which don't yet appear elsewhere in this document

:*[range]*retab :ret Command. Reformats strings of whitespace containing tabs

- Handy if you've changed '**tabstop**', or set '**expandtab**'

:visual :vi Leave Ex Mode. Get back to normal VIM editing

- Happens to me all the time (I inadvertently hit **Q** in Normal Mode)
- Ex is an old line editor that the original Vi was built on
- It is the source of all the **:** commands
- Note, **sed** is built off the same commands
 - **sed** is a *stream editor*
 - Non-interactive
 - * Driven by line numbers and regular expressions
 - Good for editing many files batch-mode
 - Even basic **sed** commands will provide a useful tool to your bag

Epilogue

I didn't touch a good deal of what Vim has to offer. This document is intended for those fairly new to Vim. It contains some information immediately useful, and some that might provide anchors for further investigation, a preview of things yet to learn.

Become proficient w/the basics, then add a feature to your repertoire, take it for a test drive. If it improves your workflow, great. If not, that's fine. Either way, pick the next feature that looks appealing, take it for a spin. There's some effort involved here, but, on balance, the effort spent will work for you, make you more productive.

Again, there's much to Vim. If you feel as though the editor should do something, it likely does. Or, perhaps, you can record macros or even write a function to do it.